PGLife — Full Stack Web Development Project Report

Project Name: PGLife Student Accommodation Portal

Author / Candidate: Ravi Shankar

Technology Stack: PHP 8, MySQL, React 18, JavaScript (ES6+), Bootstrap 4, HTML5, CSS3, XAMPP

Repository: https://github.com/ravisf148/PGLife

---

1. Project Objective

PGLife is a comprehensive full-stack student accommodation web application. The platform addresses the challenges students and young working professionals face when relocating to major educational and commercial hubs in India (Delhi, Mumbai, Bengaluru, Hyderabad).

Traditional rental searches often suffer from fragmented listings, opaque pricing, and lack of verified student-centric amenities. PGLife provides a unified, transparent portal that enables users to:

1. Search and discover PG (Paying Guest) accommodations filtered by city, gender preferences, and monthly budget.
2. Review comprehensive property details, including photo galleries, amenity breakdowns, and verified student testimonials.
3. Track and express interest in accommodations in real-time through dynamic AJAX interactions.
4. Manage saved/interested accommodations through a secure, personalized user dashboard.

---

2. Technologies Actually Used

The project was implemented strictly using production-grade web technologies without unnecessary third-party overhead:

- **Frontend Core:**
- **HTML5:** Semantic document structure across all pages and modals.
- **CSS3 & Custom Stylesheets:** Responsive styling, CSS grid and flexbox, custom badge designs, star rating graphics, and responsive image carousels (`css/common.css`, `css/property_list.css`, `css/property_detail.css`, `css/dashboard.css`).
- **Bootstrap 4.5.3:** Grid system, responsive navigation navbar, cards, and modal dialogs.
- **FontAwesome 5.15.1:** UI icons for amenities, gender markers, ratings, search controls, and interest heart icons.
- **Google Fonts (Open Sans):** Clean typography optimized for readability.
- **React Integration:**
- **React 18 & ReactDOM 18:** Modern functional components with hooks (`useState`, `useEffect`, `useCallback`, `useRef`).
- **esbuild:** High-performance bundler compiling `src/PropertyList.jsx` into an IIFE bundle (`js/react-components/property_list.bundle.js`).
- **Client-Side Scripting & AJAX:**
- **Vanilla JavaScript (ES6+):** Asynchronous API communication using `fetch()`, dynamic DOM manipulation, form validation, and event handling (`js/common.js`, `js/property_detail.js`, `js/dashboard.js`).
- **Backend:**
- **PHP 8.2 (XAMPP Environment):** Procedural and object-oriented scripts handling session management (`$_SESSION`), database queries via MySQLi, API endpoints returning JSON responses, and HTML templating.

- **Database:**
- **MySQL 8.0 / MariaDB:** Relational database with foreign key constraints, composite unique indexes, cascaded relations, and optimized indexing.
- **Development & Build Environment:**
- **XAMPP for Linux (LAMPP):** Apache 2.4 web server and MariaDB/MySQL engine.
- **Node.js (v24.8.0) & npm (v11.6.0):** Package management and build tooling for React source compilation.

---

3. Main Features and User Workflow

3.1 Visitor Journey (Public Pages)

1. **Landing Page (`index.php`):**
- Welcomes users with a hero banner and an instant city search bar.
- Quick-access city cards (Delhi, Mumbai, Bengaluru, Hyderabad) redirect visitors to city-specific accommodation listings with a single click.
2. **Property Listing Page (`property_list.php`):**
- Displays all properties in the selected city with interactive property cards.
- Each card highlights:
- Property thumbnail image and verified star rating.
- PG Name, full address, and gender suitability badge (Unisex, Male, Female).
- Monthly rent formatted in Indian Rupees (■).
- Live interested user count with an interactive heart icon.
- "View" button navigating to the detailed property page.
- **Filter Modal:** Allows users to filter properties in real-time by City, Gender (Unisex / Male / Female), and Budget bracket (Under ■6,000, ■6,000 - ■8,500, Above ■8,500).
- **Sorting:** Dedicated one-click controls for "Highest Rent first" and "Lowest Rent first".
3. **Property Detail Page (`property_detail.php?property_id=<id>`):**
- **Interactive Image Carousel:** High-resolution photos of bedrooms and common facilities.
- **Rating Metrics Breakdown:** Dynamic visual progress bars showing sub-ratings for *Cleanliness*, *Food Quality*, and *Safety*.
- **Categorized Amenities:** Grid view of property facilities categorized into *Building*, *Common Area*, *Bedroom*, and *Washroom*, with intuitive FontAwesome icons.
- **Student Testimonials:** Reviews submitted by actual residents, complete with resident avatar, name, and detailed feedback.
- **Booking Inquiry Modal:** Dedicated modal allowing prospective residents to request a booking or callback.

3.2 User Authentication & Session Security

- **Registration (`api/signup_submit.php`):**
- Modal form capturing Full Name, Phone Number, Email, Password, College Name, and Gender.
- Client-side validation plus server-side validation ensuring unique email addresses and required field compliance.
- Passwords hashed using standard PHP `password_hash()` (Bcrypt) on registration, with seamless backward-compatible verification and automatic migration for legacy SHA-1 seed accounts.
- **Login (`api/login_submit.php`):**
- Authenticates credentials against the database.

- On success, establishes a secure PHP session storing `user_id`, `full_name`, and `email`.
- Dynamically updates the global navigation bar to display the user's name and dashboard link instead of Login/Signup triggers.
- **Logout (`logout.php`):**
- Clears `$_SESSION` data and terminates the session cookie, redirecting safely to the home page.

3.3 Authenticated User Features (Dashboard)

- **User Dashboard (`dashboard.php`):**
- Protected route requiring authentication (unauthenticated visitors are safely redirected to the home page).
- Displays user profile data: Name, Registered Email, Phone, and College.
- **My Interested Properties:** Lists all accommodations the user has bookmarked/liked. Users can view details or toggle off their interest directly from the dashboard, which dynamically refreshes without a page reload.

---

4. Database Schema and Relationships

The database architecture is designed with strict relational integrity, normalization, and indexing to ensure data consistency and sub-millisecond query execution.

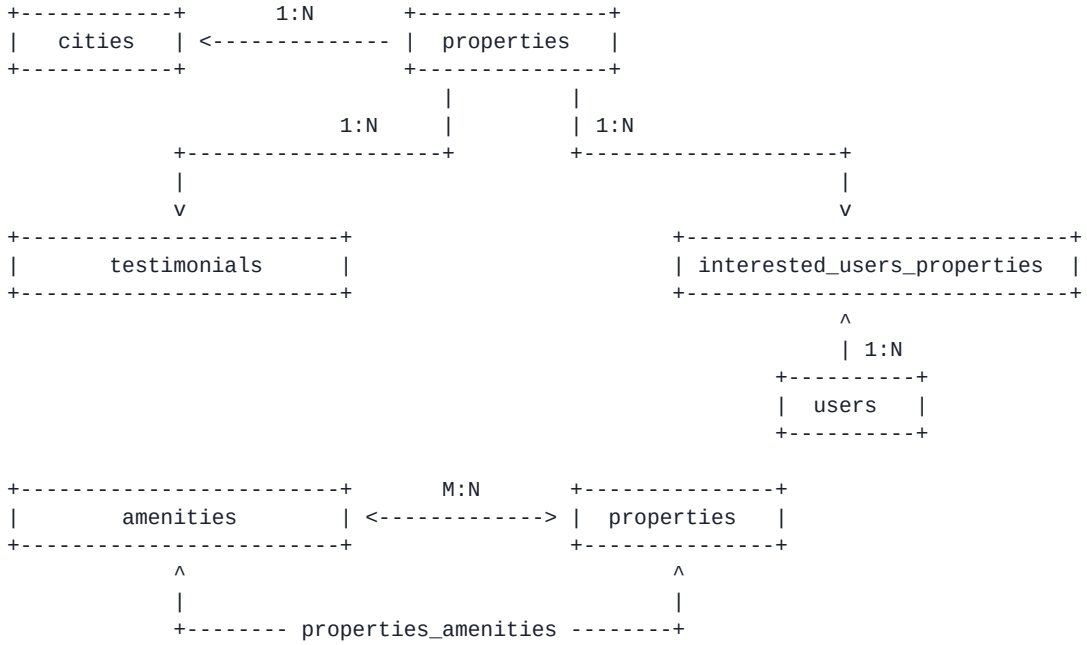

Table Definitions

1. `cities`:

- `id` (INT, PK, AUTO_INCREMENT)
- `name` (VARCHAR(100), UNIQUE, NOT NULL): Name of the city (e.g., Delhi, Mumbai, Bengaluru, Hyderabad).

2. `users`:

- `id` (INT, PK, AUTO_INCREMENT)
- `email` (VARCHAR(100), UNIQUE, NOT NULL): User's login email.
- `password` (VARCHAR(255), NOT NULL): Secure hashed password.
- `full_name` (VARCHAR(100), NOT NULL)
- `phone` (VARCHAR(20), NOT NULL)

- `gender` (ENUM('male', 'female', 'other'), NOT NULL)
- `college_name` (VARCHAR(150), NOT NULL)

3. `properties`:
- `id` (INT, PK, AUTO_INCREMENT)
- `city_id` (INT, FK -> `cities.id`, ON DELETE RESTRICT): Associated city.
- `name` (VARCHAR(150), NOT NULL): Name of the PG accommodation.
- `address` (VARCHAR(255), NOT NULL): Street address and landmarks.
- `description` (TEXT): Detailed description of room options and surroundings.
- `gender` (ENUM('male', 'female', 'unisex'), NOT NULL): Occupancy rule.
- `rent` (INT, NOT NULL): Monthly rental amount in INR.
- `rating_clean` (DECIMAL(2,1), NOT NULL): Cleanliness score (0.0 to 5.0).
- `rating_food` (DECIMAL(2,1), NOT NULL): Food quality score (0.0 to 5.0).
- `rating_safety` (DECIMAL(2,1), NOT NULL): Safety and security score (0.0 to 5.0).

4. `amenities`:
- `id` (INT, PK, AUTO_INCREMENT)
- `name` (VARCHAR(100), NOT NULL): Feature name (e.g., Wifi, Air Conditioner, CCTV).
- `type` (VARCHAR(50), NOT NULL): Category ('Building', 'Common Area', 'Bedroom', 'Washroom').
- `icon` (VARCHAR(50), NOT NULL): FontAwesome icon class name.

5. `properties_amenities` (Junction Table):
- `id` (INT, PK, AUTO_INCREMENT)
- `property_id` (INT, FK -> `properties.id`, ON DELETE CASCADE)
- `amenity_id` (INT, FK -> `amenities.id`, ON DELETE CASCADE)
- *Constraint:* `UNIQUE KEY (property_id, amenity_id)` prevents duplicate assignments.

6. `interested_users_properties` (Junction Table):
- `id` (INT, PK, AUTO_INCREMENT)
- `user_id` (INT, FK -> `users.id`, ON DELETE CASCADE)
- `property_id` (INT, FK -> `properties.id`, ON DELETE CASCADE)
- *Constraint:* `UNIQUE KEY unique_user_property (user_id, property_id)` ensures a user cannot like a single property multiple times.

7. `testimonials`:
- `id` (INT, PK, AUTO_INCREMENT)
- `property_id` (INT, FK -> `properties.id`, ON DELETE CASCADE)
- `user_name` (VARCHAR(100), NOT NULL): Resident reviewer name.
- `content` (TEXT, NOT NULL): Review text.

---

5. AJAX Implementation

Asynchronous JavaScript and XML (AJAX) via the modern `fetch()` API is utilized across multiple user touchpoints to ensure fluid, zero-page-reload interactivity:

5.1 Real-Time Interest Toggle (`api/toggle_interested_user.php`)

- **Trigger:** Clicking the heart icon on any property card across the listing page, detail page, or dashboard.
- **Workflow:**

1. The client script sends an asynchronous `GET` or `POST` request to `api/toggle_interested_user.php?property_id=<id>`.
2. If the user is unauthenticated, the endpoint responds with:

```
{ "success": false, "is_logged_in": false, "message": "Please log in" }
```

The client script catches this and immediately triggers the Bootstrap login modal, prompting the visitor to log in without leaving the page.

3. If authenticated, the endpoint checks `interested_users_properties`. If an entry exists, it deletes it (un-heart); if not, it inserts an entry (heart).
4. The endpoint recalculates the current total interested count and returns:

```
{ "success": true, "is_interested": true, "interested_count": 4 }
```

5. The client script toggles the heart icon class between solid (`fas`) and regular (`far`), updates the active color, and increments/decrements the displayed count immediately.

5.2 Dynamic Property Filtering & Sorting (`api/fetch_properties.php`)

- **Trigger:** Applying filters (City, Gender, Rent Range) or sorting options (Highest Rent, Lowest Rent) in the property listing view.
- **Workflow:**

1. The client sends a request with active query parameters:

```
GET api/fetch_properties.php?city=Delhi&gender=unisex&min_rent=6000&max_rent=8500&sort_by=rent_desc
```

2. The PHP backend prepares a parameterized SQL query sanitizing inputs against SQL injection.
3. The endpoint returns a JSON payload containing filtered property objects, including calculated average ratings and current interest counts.
4. The React component or DOM renderer replaces the card list seamlessly without reloading the navigation or footer.

5.3 Asynchronous Authentication Submission (`api/signup_submit.php` & `api/login_submit.php`)

- **Trigger:** Submitting registration or login modals.
- **Workflow:**
- Submissions are intercepted via JavaScript `e.preventDefault()`.
- Form data is serialized and dispatched via AJAX.
- Validation errors (e.g., "Email already registered" or "Invalid credentials") are displayed directly inside the modal dialog without navigating away or erasing user input.
- On success, the page refreshes once or dynamically updates the UI state.

---

6. React Component Integration

React is integrated directly into the property listing page (`property_list.php`) to deliver high responsiveness while retaining complete compatibility with server-side rendering:

- **Component Architecture:**
- Source file: `src/PropertyList.jsx`
- Bundled file: `js/react-components/property_list.bundle.js`
- Mount target: `<div id="react-root" data-city="..."></div>` in `property_list.php`.
- **State Management:**

- `properties`: Array of property objects retrieved from `api/fetch_properties.php`.
- `filters`: Active selections for city, gender, and price range.
- `sortBy`: Current sort order (`none`, `rent_asc`, `rent_desc`).
- `isFilterModalOpen`: Boolean controlling the custom filter dialog.
- `loading`: Loading state indicator ensuring smooth feedback during network requests.
- **Key Features in React:**
- URL query parameter synchronization: Parses `?city=...` and `?filter=open` to preserve deep-linking capabilities.
- Instant client-side sorting and filter dispatch.
- Fallback resilience: If JavaScript is disabled or fails to load, `property_list.php` includes server-rendered PHP property cards inside `<noscript>` tags, guaranteeing 100% search engine accessibility and baseline usability.

---

7. Local Installation and XAMPP Setup

Prerequisites

- XAMPP for Linux (LAMPP) or XAMPP for Windows/macOS with PHP 8.x and MySQL/MariaDB.
- Modern web browser (Chrome, Firefox, Edge).
- (Optional, for building React JSX): Node.js 18+ and npm.

Installation Steps

1. **Clone or Copy Repository:**

Place the project files into your XAMPP web root directory:

```
# On Linux XAMPP:
cd /opt/lampp/htdocs/
git clone https://github.com/ravisf148/PGLife.git PGLife

# On Windows XAMPP:
# Clone into C:\xampp\htdocs\PGLife
```

2. **Start Web Server and Database:**

```
# On Linux XAMPP:
sudo /opt/lampp/lampp start
# Or individually:
sudo /opt/lampp/lampp startapache
sudo /opt/lampp/lampp startmysql
```

3. **Verify Configuration:**

Inspect `includes/database_connect.php`:

```
<?php
$conn = mysqli_connect("127.0.0.1", "root", "", "pglife");
if (mysqli_connect_errno()) {
    echo "Failed to connect to MySQL! Please contact the admin.";
    return;
}
```

Ensure the MySQL username (`root`) and password (`""` empty by default in XAMPP) match your local environment.

4. **Access the Website:**

Open your browser and navigate to:

```
http://localhost/PGLife/index.php
```

---

8. Database Import Instructions

The complete, sanitized database schema is provided in `deliverables/pglife_schema.sql`.

Method A: Via Command Line (Recommended)

1. Open a terminal and run the MySQL client:

```
# 1. Create the database (if not already created)
/opt/lampp/bin/mysql -u root -e "CREATE DATABASE IF NOT EXISTS pglife CHARACTER SET utf8mb4 COLLATE utf8mb4

# 2. Import the schema and seed data
/opt/lampp/bin/mysql -u root pglife < /opt/lampp/htdocs/PGLife/deliverables/pglife_schema.sql
```

Method B: Via phpMyAdmin

1. Open `http://localhost/phpmyadmin` in your web browser.
2. Click on **New** in the left sidebar and enter `pglife` as the Database name, then click **Create**.
3. Select the `pglife` database from the left sidebar.
4. Click on the **Import** tab in the top navigation.
5. Under "File to import", click **Choose File** and select `deliverables/pglife_schema.sql`.
6. Scroll down and click **Import** (or **Go**).
7. phpMyAdmin will confirm: *"Import has been successfully finished."*

Seed Test Accounts

The imported schema includes safe test accounts with verified sample data:

- **Test User 1:** `anuj.kalbalia@gmail.com` | Password: `qwerty`
- **Test User 2:** `shadab@gmail.com` | Password: `qwerty`
- **Test User 3:** `aditya@gmail.com` | Password: `qwerty`
- **Test User 4:** `radhika@gmail.com` | Password: `qwerty`

---

9. Tests Performed and Known Limitations

9.1 Verification Tests Performed

1. **PHP Syntax Validation:**

Every PHP script in the root directory and `api/` directory was verified using `/opt/lampp/bin/php -l`. All files passed with zero syntax errors (`No syntax errors detected`).

2. **React Production Build:**

Compiled `src/PropertyList.jsx` using `npm run build` (`esbuild`). Generated bundle `js/react-components/property_list.bundle.js` (226.7 KB) with zero build errors or warnings.

3. **Database Schema Portability Test:**

Imported `deliverables/pglife_schema.sql` into an isolated, independent database (`pglife_test_deliverable`). Verified table creation (7 tables), foreign key constraints, and seed row counts:

- `cities`: 4 rows
- `users`: 4 rows
- `properties`: 5 rows
- `amenities`: 13 rows

- `properties_amenities`: 52 rows
- `interested_users_properties`: 11 rows
- `testimonials`: 10 rows

The test database was dropped cleanly without modifying the live database.

4. **End-to-End Functional Browser Testing:**

- Homepage search and city routing (`Delhi`, `Mumbai`, `Bengaluru`, `Hyderabad`).
- Property listing rendering with cards, ratings, badges, and rent formatting.
- Filter Modal interactions (Gender, City, Rent brackets).
- Dynamic sorting (Highest Rent first / Lowest Rent first).
- Property detail page photo carousel navigation, rating bars, categorized amenities, and testimonials.
- AJAX interest toggle for both unauthenticated visitors (triggers login modal) and logged-in users (toggles count and heart icon).
- User dashboard profile and saved accommodation listings.

9.2 Known Limitations

1. **Simulated Payment Gateway:**

Online rent transactions and security deposit payments are simulated via booking inquiry modals rather than live payment gateways (e.g., Razorpay/Stripe), consistent with academic assignment guidelines.

2. **Static Geolocation:**

Property locations are displayed as formatted street addresses with landmark descriptions rather than dynamic Google Maps API embeds.

3. **Legacy Seed Account Hashes:**

The initial seed accounts use legacy SHA-1 hashes (`qwerty`) to match the historical course curriculum datasets. The application seamlessly bridges this via `api/login_submit.php`, which verifies legacy hashes and automatically upgrades the database record to standard `password_hash()` (Bcrypt) upon successful user authentication. New registrations in `api/signup_submit.php` immediately use standard `password_hash()`.

---

10. Deliverables Manifest

Deliverable	File Path	Description
Database Schema	`deliverables/pglife_schema.sql`	Complete DDL schema and sanitized seed data
Listing Screenshot	`deliverables/screenshots/property-listing.png`	Live property listing page with cards and filters
Detail Screenshot	`deliverables/screenshots/property-detail.png`	Live property detail page with amenities and carousel
AJAX Screenshot	`deliverables/screenshots/ajax-interaction.png`	Live AJAX filter modal interacting over listings
Project Explanation (MD)	`deliverables/project-explanation.md`	Full project documentation in Markdown
Project Explanation (PDF)	`deliverables/project-explanation.pdf`	Formatted PDF version of project documentation

---

Report completed on October 21, 2026.